Azərbaycan Respublikası Elm və Təhsil Nazirliyi

Qarabağ Universiteti

Mühəndislik Fakültəsi

Verilənlərin Strukturu və Alqoritmlər

Mexatronika və robototexnika mühəndisliyi və Kompüter mühəndisliyi ixtisası üzrə bakalavr proqramının tələbələri üçün.

Laboratoriya işi № 2

Tələbənin adı və soyadı: Nazilə Gülməmmədova

Qrup: KOM25

Müəllim: Şamil Hümbətov

Tarix: 11 oktyabr 2026

2026

Laboratoriya işi №1

Singly Linked List-in reallaşdırılması və tətbiqi

Məqsəd

Bu laboratoriya işinin məqsədi:

- Linked List verilənlər strukturunun iş prinsipini öyrənmək;
- Düyün (Node) anlayışını mənimsəmək;
- Siyahıya element əlavə etmə və silmə əməliyyatlarını reallaşdırmaq;
- Linked List və adi massiv (array/list) arasındakı fərqləri anlamaq.

Nəzəri hissə

Linked List düyünlərdən ibarətdir.

Hər düyün iki hissədən ibarətdir:

```
+-------+------+
| data  | next |
+-------+------+
```

məsələn

10 -> 20 -> 30 -> NULL

Burada:

- `10`, `20`, `30` verilənlərdir
- `next` növbəti düyünün ünvanını göstərir
- `NULL` siyahının sonunu bildirir

Tapşırıq 1. Node sinfinin yaradılması

Aşağıdakı strukturu yaradın:

class Node:

def **init**(self, data):

self.data = data

self.next = None

Test:

Node(15)

Həll:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
class LinkedList:
    def __init__(self):
        self.head = None
```

Tapşırıq 2. Siyahının sonuna element əlavə edilməsi

Aşağıdakı siyahını formalaşdırın:

10 → 20 → 30 → 40

və çap edin.

Nəticə:

10 -> 20 -> 30 -> 40 -> None

Həll:

```
 def insert_at_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        current = self.head
        while current.next is not None:

            current = current.next
        current.next = new_node
```

Tapşırıq 3. Siyahının əvvəlinə element əlavə edilməsi

Verilmiş siyahı:

20 → 30 → 40

başlanğıca 10 əlavə edin.

Nəticə:

10 → 20 → 30 → 40

Həll:

```
def insert_at_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
```

Tapşırıq 4. Verilmiş elementdən sonra əlavə etmə

Verilmiş siyahı:

10 → 20 → 40

20-dən sonra 30 əlavə edin.

Nəticə:

10 → 20 → 30 → 40

Həll:

```
def insert_after(self, target, data):
        current = self.head
        while current is not None:
            if current.data == target:
                new_node = Node(data)
                new_node.next = current.next
                current.next = new_node
                return
            current = current.next
        print("Element tapılmadı:", target)
```

Tapşırıq 5. Element silmə

Verilmiş siyahı:

10 → 20 → 30 → 40 → 50

30 elementini silin.

Nəticə:

10 → 20 → 40 → 50

Həll:

```
def delete(self, value):
        if self.head is None:
            return
        # silinən element birinci düyündürsə
        if self.head.data == value:
            self.head = self.head.next
            return
        current = self.head
        while current.next is not None:
            if current.next.data == value:
                current.next = current.next.next    # düyünü atlayırıq
                return
            current = current.next
        print("Silinəcək element tapılmadı:", value)
```

Tapşırıq 6. Axtarış

Funksiya yazın:

search(value)

Əgər element siyahıda varsa:

Element tapıldı.

əks halda

Element tapılmadı.

Həll:

```
def search(self, value):
        current = self.head
        while current is not None:
            if current.data == value:
                print("Element tapıldı.")
                return True
            current = current.next
        print("Element tapılmadı.")
        return False
```

Tapşırıq 7. Uzunluğun hesablanması

Linked List-də neçə element olduğunu hesablayın.

Misal:

11 → 22 → 33 → 44 → 55

Nəticə:

List length = 5

Həll:

```
def length(self):
        count = 0
        current = self.head
        while current is not None:
            count += 1
            current = current.next
        return count
```

Tapşırıq 8. Maksimum və minimum element

Aşağıdakı siyahı üçün:

23 → 8 → 91 → 15 → 42

tapın:

- maksimum element
- minimum element

Həll:

```
def find_max(self):
        if self.head is None:
            return None
        maximum = self.head.data
        current = self.head.next
        while current is not None:
            if current.data > maximum:
                maximum = current.data
            current = current.next
        return maximum

    def find_min(self):
        if self.head is None:
            return None
        minimum = self.head.data
        current = self.head.next
        while current is not None:
            if current.data < minimum:
                minimum = current.data
            current = current.next
        return minimum
```

Tapşırıq 9. Siyahını tərs çevirmək

Verilmiş:

10 → 20 → 30 → 40 → 50

Nəticə:

50 → 40 → 30 → 20 → 10

Məhdudiyyət:

❌ Yeni Linked List yaratmaq olmaz.

Həll:

```
def reverse(self):
        previous = None
        current = self.head
        while current is not None:
            next_node = current.next
            current.next = previous
            previous = current
            current = next_node
        self.head = previous
```

```
    def display(self):
        current = self.head
        text = ""
        while current is not None:
            text += str(current.data) + " -> "
            current = current.next
        print(text + "None")
```

Tapşırıq 10 (Challenge)

Tələbə qeydiyyat sistemi hazırlayın.

Hər düyündə:

student_id

student_name

gpa

saxlanılsın.

Aşağıdakı əməliyyatları təmin edin:

1. Tələbə əlavə et
2. Tələbə sil
3. Bütün tələbələri göstər
4. GPA > 80 olanları göstər
5. Ada görə axtarış et
6. Siyahıdakı tələbələrin sayını göstər

Həll:

```
class StudentNode:
    def __init__(self, student_id, student_name, gpa):
        self.student_id = student_id
        self.student_name = student_name
        self.gpa = gpa
        self.next = None

class StudentList:
    def __init__(self):
        self.head = None

    def add_student(self, student_id, student_name, gpa):
        new_node = StudentNode(student_id, student_name, gpa)
        if self.head is None:
            self.head = new_node
            return
```

```
        current = self.head
        while current.next is not None:
            current = current.next
        current.next = new_node

    def delete_student(self, student_id):
        if self.head is None:
            return
        if self.head.student_id == student_id:
            self.head = self.head.next
            return
        current = self.head
        while current.next is not None:
            if current.next.student_id == student_id:
                current.next = current.next.next
                return
            current = current.next
        print("Tələbə tapılmadı:", student_id)

    def show_all(self):
        current = self.head
        while current is not None:
            print(current.student_id, current.student_name, current.gpa)
            current = current.next

    def show_high_gpa(self):
        current = self.head
        while current is not None:
            if current.gpa > 80:
                print(current.student_id, current.student_name, current.gpa)
            current = current.next

    def search_by_name(self, name):
        current = self.head
        while current is not None:
            if current.student_name == name:
                print("Tapıldı:", current.student_id, current.student_name,
current.gpa)
                return
            current = current.next
        print("Tələbə tapılmadı:", name)

    def count(self):
        total = 0
        current = self.head
        while current is not None:
```

```
            total += 1
            current = current.next
        return total
```

Bonus tapşırıq (+10 bal)

Singly Linked List və Doubly Linked List müqayisəsi

Aşağıdakıları araşdırın:

- Struktur fərqləri
- Üstünlüklər
- Çatışmazlıqlar
- Yaddaş sərfi
- İrəli və geri traversing

Nəticələri 1 səhifəlik qısa hesabat şəklində təqdim edin.

Bonus sual(hesabat):

Singly Linked List və Doubly Linked List müqayisəsi

Struktur fərqləri. Singly Linked List-də hər düyündə data və next var. Doubly Linked List-də əlavə olaraq prev var (əvvəlki düyünə istinad).

Üstünlüklər.

- Singly: sadədir, az yaddaş işlədir, reallaşdırması asandır.
- Doubly: həm irəli, həm geri gəzmək olur; düyünü silmək asandır, çünki əvvəlki düyün prev ilə məlumdur; sona əlavə etmək tail ilə sürətlidir.

Çatışmazlıqlar.

- Singly: yalnız irəli gəzmək olur; silmək üçün əvvəlki düyünü tapmaq lazımdır.
- Doubly: hər düyündə əlavə istinad var, kod daha mürəkkəbdir və səhv etmə ehtimalı artır.

Yaddaş sərfi. Doubly Linked List hər düyün üçün bir əlavə istinad (prev) saxlayır, ona görə Singly-dən çox yaddaş işlədir.

İrəli və geri traversing. Singly-də yalnız irəli (head -> tail). Doubly-də həm irəli, həm geri (tail -> head).

Nəticə. Yaddaş vacibdirsə və yalnız irəli gəzmək kifayətdirsə, Singly seçilir. Geri hərəkət və tez silmə lazımdırsa, Doubly seçilir.

Qiymətləndirmə

- Tapşırıq 1-3: 20 bal
- Tapşırıq 4-5: 20 bal

- Tapşırıq 6-7: 15 bal
- Tapşırıq 8-9: 25 bal
- Tapşırıq 10: 20 bal

Bonus: +10 bal



